

Unit 2

Experiment 5 - Spring Boot REST API Design & Exception Handling

Aim:

Design RESTful APIs using Spring Boot that enforce consistent response structures, handle exceptions globally, log request details, and support secure cross-origin communication.

Description:

Modern web applications rely heavily on backend services to manage data, enforce business logic, and ensure secure communication between systems. As applications scale, it becomes essential to design APIs that are **consistent, robust, and maintainable, while also ensuring reliability, observability, and security.**

This experiment focuses on building RESTful APIs using **Spring Boot**, incorporating **standardized response structures, validation mechanisms, global exception handling, and logging strategies.** It also introduces **filters, interceptors, and correlation-based tracing**, which are critical for debugging and monitoring in production systems.

Theoretical Explanation:

1. REST API Design Principles

REST (Representational State Transfer) is an architectural style for designing scalable web services.

Core Principles:

- Statelessness – Each request must contain all necessary information. The server does not store session state.
- Resource-Based Design – APIs represent resources using URIs:

 /api/posts
 /api/posts/{id}
- HTTP Methods Semantics
 - GET → Retrieve data
 - POST → Create resource
 - PUT → Update resource
 - DELETE → Remove resource
- Uniform Interface – Consistent request/response structure across endpoints

Academic Insight:

REST is based on **distributed system constraints**, enabling **scalability, loose coupling, and interoperability** across heterogeneous systems.

2. Spring Boot Architecture

Spring Boot follows a **layered** architecture pattern:

Controller → Service → Repository → Database

Code Example: Controller Layer

```
@RestController
@RequestMapping("/api/posts")
public class PostController {

    private final PostService postService;

    public PostController(PostService postService) {
        this.postService = postService;
    }

    @PostMapping
    public ResponseEntity<?> createPost(@RequestBody PostRequest request) {
        return ResponseEntity.ok(postService.createPost(request));
    }
}
```

Explanation:

- Handles HTTP requests
- Delegates logic to service layer
- Maintains separation of concerns

Academic Insight:

This reflects the Separation of Concerns principle, improving modularity and maintainability.

3. **Bean Validation (Data Integrity Enforcement)**

Code Example

```
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;

public class PostRequest {

    @NotBlank(message = "Content must not be empty")
    private String content;

    @Size(max = 280, message = "Content exceeds limit")
    private String text;
}
```

Explanation:

- Ensures input correctness before processing
- Prevents invalid data propagation
- Improves API contract reliability

Academic Insight:

Bean Validation aligns with **Design by Contract (DbC)**, where inputs must satisfy predefined constraints.

4. **API Response Standardization**

Code Example

```
public class ApiResponse<T> {  
  
    private String status;  
    private String message;  
    private T data;  
  
    public ApiResponse(String status, String message, T data) {  
        this.status = status;  
        this.message = message;  
        this.data = data;  
    }  
}
```

Usage Example

```
return ResponseEntity.ok(  
    new ApiResponse<>("success", "Post created", post)  
);
```

Explanation:

- Uniform response structure
- Easier frontend integration
- Simplifies debugging

Academic Insight:

Standardization improves **interface consistency**, a key aspect of API usability and maintainability.

5. CORS (Cross-Origin Resource Sharing)

Code Example

```
@CrossOrigin(origins = "http://localhost:3000")  
@RestController  
public class PostController { }
```

Explanation:

- Allows frontend (different origin) to access APIs
- Prevents unauthorized cross-origin requests

Academic Insight:

CORS is part of **browser security models**, preventing malicious cross-site interactions.

6. Filters & Interceptors (Middleware Architecture)

Code Example: Logging Filter

```
@Component  
public class LoggingFilter extends OncePerRequestFilter {  
  
    @Override  
    protected void doFilterInternal(HttpServletRequest request,  
        HttpServletResponse response,  
        FilterChain chain) throws ServletException, IOException {  
        // Logging logic  
        chain.doFilter(request, response);  
    }  
}
```

```

        HttpServletResponse response,
        FilterChain filterChain)
        throws ServletException, IOException {

    long startTime = System.currentTimeMillis();

    filterChain.doFilter(request, response);

    long duration = System.currentTimeMillis() - startTime;
    System.out.println("Request: " + request.getRequestURI() +
        " took " + duration + " ms");
    }
}

```

Explanation:

- Intercepts requests before reaching controller
- Logs request lifecycle
- Measures performance

Academic Insight:

Filters implement the **Middleware Pattern**, widely used in web servers and distributed systems.

7. Global Exception Handling (@ControllerAdvice)

Code Example

```

@ControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<?> handleValidationException(
        MethodArgumentNotValidException ex) {

        return ResponseEntity.badRequest().body("Validation failed");
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<?> handleGenericException(Exception ex) {

        return ResponseEntity
            .status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body("Internal Server Error");
    }
}

```

Explanation:

- Centralized error handling
- Prevents duplicate error logic
- Ensures consistent error responses

Academic Insight:

This aligns with cross-cutting concerns handled using **Aspect-Oriented Programming (AOP)**.

8. Logging & Monitoring (MDC and Correlation ID)

Code Example

```
import org.slf4j.MDC;
import java.util.UUID;

String correlationId = UUID.randomUUID().toString();
MDC.put("correlationId", correlationId);
```

Logging Configuration Example

```
log.info("Processing request for Post ID: {}", postId);
```

Explanation:

- Each request gets a unique ID
- Helps trace logs across system
- Useful in debugging distributed systems

Academic Insight:

This supports observability principles, including:

- Logging
- Tracing
- Monitoring

9. Advanced: Interceptor for Correlation ID

Code Example

```
@Component
public class CorrelationInterceptor implements HandlerInterceptor {

    @Override
    public boolean preHandle(HttpServletRequest request,
                             HttpServletResponse response,
                             Object handler) {

        String correlationId = UUID.randomUUID().toString();
        MDC.put("correlationId", correlationId);
        return true;
    }

    @Override
    public void afterCompletion(HttpServletRequest request,
                               HttpServletResponse response,
                               Object handler,
                               Exception ex) {

        MDC.clear();
    }
}
```

Explanation:

- Adds tracing capability
- Cleans up after request completion

Experiment 5.1: REST API Design

Conceptual Focus:

- REST architecture principles
- Layered system design
- Request validation
- API standardization
- Secure communication

Implementation Overview:

- Create CRUD APIs
- Apply validation
- Structure responses
- Configure CORS

Experiment 5.2: Exception Handling & Logging

Conceptual Focus:

- Centralized exception handling
- Middleware architecture
- Logging and tracing
- Observability

Implementation Overview:

- Implement `@ControllerAdvice`
- Add logging filter
- Use MDC for correlation ID
- Standardize error responses

Integration Perspective

The REST API design module establishes structured and validated endpoints for data operations, while exception handling and logging mechanisms ensure robustness, traceability, and maintainability. Together, they form a scalable and production-ready backend system capable of supporting complex frontend interactions.

Assignments

Assignment 1 (REST API Implementation)

- Create CRUD APIs for posts
- Implement scheduling endpoint

Assignment 2 (Validation)

- Add Bean Validation
- Test invalid inputs

Assignment 3 (Logging Filter)

- Log request URI and execution time

Assignment 4 (Exception Handling)

- Implement global exception handler
- Handle validation and generic exceptions

Assignment 5 (Advanced: Correlation ID)

- Add correlation ID using MDC
- Verify logs contain request trace

Key Takeaways

- REST APIs enable scalable communication
- Layered architecture improves maintainability
- Validation ensures data integrity
- Exception handling improves robustness
- Logging enhances observability

Future Scope

The concepts covered in this experiment can be extended to build **enterprise-grade backend systems** by integrating microservices architecture, API gateways, and advanced security mechanisms such as OAuth2 and JWT-based authorization. Additionally, incorporating distributed tracing tools, centralized logging systems, and cloud-based deployment strategies will further enhance scalability, observability, and reliability in real-world applications.